# SIMATS ENGINEERING
## ASSESSMENT TOOL 3

**COURSE CODE: CSA37** **COURSE NAME: Software Testing**

**COURSE OUTCOME COVERED**

***CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow.*** *(BL3)*

*Assessment Tool Weightage: 20%*

**Duration :60 Mins**
**On Class**
**QUESTIONS: 5** **Total Marks:50**

Annexure A

## Error Analysis Task

***Code of Conduct:***

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

Annexure A

# Error Analysis Task

**Q1. Error Analysis of an Online Examination System**
**Tools:** Selenium WebDriver, Chrome DevTools, Excel

A university online examination system has reported errors in student registration, attendance eligibility, and examination access. Some students with less than 75% attendance can access examinations, while eligible students are occasionally rejected.

a) Analyze the reported errors and identify the expected and actual behavior using systematic test design strategies.

b) Apply Equivalence Class Partitioning, Boundary Value Analysis, and Random Testing to identify input-related errors.

c) Design black-box test cases using decision tables and state transition testing to identify errors in examination eligibility and access.

d) Execute the test cases using Selenium WebDriver and document the observed errors with screenshots.

e) Evaluate test adequacy by identifying untested requirements, prioritizing high-risk errors, and recommending additional test cases.

**Q2. Error Analysis of a Student Grade Calculator**
**Tools:** Java, JUnit, Maven, JaCoCo

A university grade-calculation application produces incorrect grades for marks near grade boundaries. The source code is available, but the existing test suite does not identify all reported errors.

a) Analyze the source code to identify incorrect conditions, missing validations, and possible logical errors.

b) Apply Equivalence Class Partitioning and Boundary Value Analysis to identify errors in mark validation and grade assignment.

c) Construct a Control Flow Graph (CFG), identify independent paths, and calculate cyclomatic complexity to determine the test paths required for error detection.

d) Develop JUnit test cases and use JaCoCo to analyze statement and branch coverage, identifying uncovered code that may contain defects.

e) Apply mutation testing and evaluate whether the test suite is adequate to detect logical errors and verify the corrected program.

**Q3. Error Analysis of an Online Banking Application**
**Tools:** Postman, Swagger/OpenAPI, Excel

An online banking application occasionally processes invalid fund-transfer requests, displays inconsistent error messages, and reports incorrect transaction statuses when network failures occur.

a) Analyze the API requests and responses to identify validation errors, transaction errors, and inconsistencies between expected and actual behavior.

b) Apply black-box test design techniques, including Equivalence Class Partitioning, Boundary Value Analysis, Random Testing, and decision tables, to identify transaction-related errors.

c) Design use-case, state transition, and pairwise test cases to investigate errors involving authentication, transaction status, network type, browser, and repeated transfer requests.

d) Use Postman to execute the test cases and document the errors, including request details, response codes, actual results, and expected results.

e) Evaluate test adequacy using requirement traceability and risk coverage. Explain which internal code-coverage measures cannot be verified when the source code is unavailable.

**Q4. Error Analysis of a Commercial Off-The-Shelf (COTS) Learning Management System**
**Tools:** Selenium WebDriver, Apache JMeter, Chrome DevTools

A university purchases a third-party Learning Management System (LMS). Users report assignment-upload failures, browser compatibility problems, and slow responses during peak usage. The source code is not available.

a) Reverse engineer the observable application behavior and identify possible errors in login, course registration, assignment submission, and result viewing.

b) Apply black-box test design techniques, including Equivalence Class Partitioning, Boundary Value Analysis, exploratory testing, and error guessing, to identify functional errors.

c) Use Selenium WebDriver to investigate functional and compatibility errors, and use Apache JMeter to analyze performance-related failures against predefined acceptance criteria.

d) Analyze the limitations of COTS testing and explain why internal Control Flow Graphs, independent execution paths, and source-code coverage cannot normally be measured without suitable source or instrumentation access.

e) Evaluate test adequacy using functional coverage, compatibility results, performance results, and risk analysis. Recommend whether the application satisfies the defined acceptance criteria.

**Q5. Error Analysis of a Student Result-Processing Program**
**Tools:** Python, pytest, Coverage.py, Ruff or Pylint

A student result-processing program calculates totals and grades for multiple students. Errors have been reported due to incorrect loop execution, improper variable initialization, invalid mark acceptance, and incorrect pass/fail decisions.

a) Analyze the program logic and identify potential errors involving conditions, loops, variable definitions, and variable uses.

b) Apply boundary-value testing, equivalence-class testing, and error guessing to identify errors involving marks, student counts, subject counts, and invalid inputs.

c) Construct a Control Flow Graph, calculate cyclomatic complexity, and design test cases for independent paths, loop boundaries, and important decision outcomes.

d) Use pytest and Coverage.py to execute the test suite, identify uncovered statements and branches, and investigate whether insufficient coverage is hiding errors. Apply data-flow testing and mutation testing to identify additional logical defects.

e) Evaluate the adequacy of the final test suite using statement coverage, branch coverage, path considerations, data-flow coverage, and mutation results. Recommend further tests before releasing the program.

**RUBRICS FOR CALCULATION:**

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Error Identification | 20% | Accurately identifies all errors with clear understanding of issues | Identifies most errors with minor omissions | Identifies limited errors; some are missed | Fails to identify key errors |
| Root Cause Analysis | 30% | Clearly explains underlying causes with strong technical reasoning | Explains causes with moderate clarity | Partial explanation; weak reasoning | Unable to identify causes of errors |
| Correction & Justification | 25% | Provides correct solutions with strong justification and logic | Provides correct corrections with some justification | Corrections are partially correct or weakly justified | Incorrect or no corrections provided |
| Application of Concepts | 15% | Effectively applies relevant concepts to analyze and resolve errors | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply concepts |
| Clarity & Presentation | 10% | Presents analysis clearly, logically, and systematically | Generally clear with minor issues | Lacks clarity or proper structure | Poor presentation and difficult to understand |

# SIMATS ENGINEERING
## CSA37 — SOFTWARE TESTING
### ASSESSMENT TOOL 3

# ERROR ANALYSIS TASK

Comprehensive answers to Questions 1–5

Name: ARAVINDH R
Register No.: 192312024

## Q1. Error Analysis of an Online Examination System

### a) Expected versus actual behavior and systematic analysis

The reported symptoms indicate a mismatch between the eligibility rule, registration data, and the examination-access state. The first step is to reproduce each symptom using a controlled account and preserve timestamped logs, input data, browser version, request/response details, and screenshots.

| ID / symptom | Expected behavior | Reported / possible actual behavior | Likely root cause | Priority |
|---|---|---|---|---|
| Q1-E1: attendance below 75% | Block exam launch and show a clear eligibility message. | Some students below 75% can enter. | Comparison uses > / wrong field / stale attendance cache / client-only check. | Critical |
| Q1-E2: attendance exactly 75% | Allow if the rule is ≥75% and all other conditions pass. | Could be rejected at the boundary. | Strict >75 comparison, rounding mismatch, or inconsistent percentage calculation. | High |
| Q1-E3: eligible student rejected | Permit exam when registration, schedule, and attendance checks pass. | Intermittent denial. | Stale cache, wrong student-course mapping, timezone/window mismatch, race condition. | High |
| Q1-E4: duplicate registration | Maintain one valid registration per student/course/exam. | Duplicate or mismatched record possible. | Missing uniqueness constraint or non-atomic registration update. | High |
| Q1-E5: refresh/back/reopen exam | Resume the same attempt according to policy; never create an unintended extra attempt. | Attempt may reset or duplicate. | Weak state management and missing idempotency/attempt lock. | High |
| Q1-E6: unauthorized direct URL | Reject unauthenticated or ineligible requests server-side. | UI may be bypassed if checks are client-only. | Authorization/eligibility enforced only in browser. | Critical |

### b) Equivalence Class Partitioning (ECP), Boundary Value Analysis (BVA), random testing

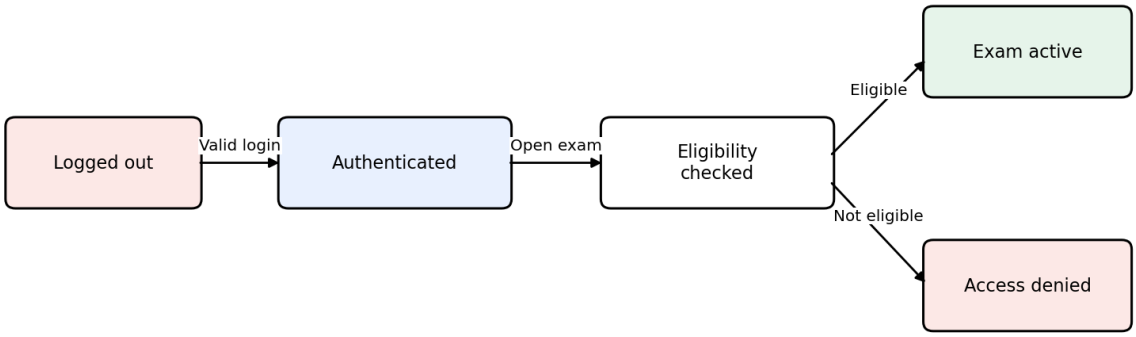
| Technique / field | Classes or values | Expected result |
|---|---|---|
| ECP: attendance % | Invalid: <0, >100, null, non-numeric; ineligible: 0–74.99; eligible: 75–100 | Reject invalid values; block ineligible; allow eligible if all other checks pass. |
| BVA: attendance | 74.99%, 75.00%, 75.01% (also 74%, 76%) | 74.99 deny; 75.00 allow; 75.01 allow. |
| ECP: registration | Not registered / registered / duplicate / cancelled | Only one active valid registration permits access. |
| ECP: exam window | Before start / within window / after end | Deny before and after; allow during the configured window. |
| BVA: time | One second before start; exact start; exact end; one second after end | Follow the explicitly defined inclusive/exclusive time rule. |

| | | |
|---|---|---|
| Random testing | Generate attendance in [0,100], malformed values, nulls, duplicate submissions, rapid refreshes, and simultaneous requests using a fixed seed | No invalid value may bypass checks; record seed and failing input for reproducibility. |

## c) Black-box decision table and state-transition testing

| Rule | Authenticated? | Registered? | Attendance ≥75%? | Within exam window? | Expected action |
|---|---|---|---|---|---|
| R1 | No | — | — | — | Deny; prompt sign-in |
| R2 | Yes | No | — | — | Deny; registration required |
| R3 | Yes | Yes | No | — | Deny; attendance requirement not met |
| R4 | Yes | Yes | Yes | No | Deny; exam not open |
| R5 | Yes | Yes | Yes | Yes | Grant access and create/resume one attempt |

**Q1 — Examination Access State Transitions**




State-transition test cases: (1) Logged out → login with valid credentials → authenticated; (2) authenticated but unregistered → exam URL → access denied; (3) registered with 74.99% attendance → denied; (4) registered with 75.00% → allowed if the exam window is open; (5) active attempt → refresh/network interruption → same attempt resumes without duplicate; (6) submitted attempt → reopening must follow policy and must not silently permit a second submission.

## d) Selenium execution plan and sample evidence

Because the actual system URL and credentials were not supplied, the following is a runnable template structure, not a claim of a live execution. Configure a test environment and stable locators before running.

```
# Python example using Selenium 4 (adapt selectors and URL)
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
try:
    driver.get("https://TEST-ENV.example.edu/login")
    driver.find_element(By.ID, "username").send_keys("TEST_STUDENT_74")
    driver.find_element(By.ID, "password").send_keys("TEST_PASSWORD")
    driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
    WebDriverWait(driver, 10).until(
        EC.element_to_be_clickable((By.ID, "open-exam"))
    ).click()
    # Save evidence on both pass and fail; redact personal data.
    driver.save_screenshot("Q1_exam_access.png")
finally:
    driver.quit()
```

**Q1 — Sample Test Execution Report (Illustrative)**

| Test ID | Scenario | Expected | Observed (illustrative) | Status |
|---|---|---|---|---|
| REG-04 | Attendance = 74.99% | Deny exam access | Access granted | FAIL |
| REG-05 | Attendance = 75.00% | Allow if other rules pass | Access denied | FAIL |
| AUTH-02 | Invalid password | Show generic error | Login blocked | PASS |
| EXAM-08 | Refresh during active exam | Resume same attempt safely | New attempt created | FAIL |

These are simulated examples, not screenshots from a live university system.

The pictured report is deliberately labelled illustrative. For genuine evidence, capture the browser state with Selenium save_screenshot(), preserve the test ID and timestamp, and attach a sanitized Chrome DevTools Network/HAR record. Do not include real student passwords or personally identifiable data.

### e) Adequacy, risks, and additional tests

- Trace every requirement to one or more test IDs: authentication, registration, attendance calculation, exam window, attempt creation/resume, submission, and audit logging.
- Prioritize eligibility bypass and unauthorized direct-URL access as critical because they affect fairness, compliance, and exam integrity.
- Add API-level tests so server-side authorization is checked independently of browser controls.
- Test concurrent login/launch requests, duplicate clicks, stale attendance updates, cache invalidation, daylight-saving/timezone boundaries, and database failure.
- Test accessibility and browser compatibility on the supported browser/OS matrix.
- Exit criterion: all critical/high-risk tests pass; no unresolved eligibility bypass; evidence and defects are traceable to requirements.

## Q2. Error Analysis of a Student Grade Calculator

### a) Source-level defect analysis

The brief says source code is available but does not attach it. The following are common defect hypotheses to verify against the real implementation, not confirmed line-level findings.

| Potential defect | Root cause / effect | Correction |
|---|---|---|
| Wrong boundary operator | Using mark > 90 instead of mark ≥ 90 can misgrade 90. | Use explicit inclusive bands and test every threshold. |
| Overlapping / gapped grade bands | Independent if-statements or inconsistent ranges can overwrite or omit grades. | Use ordered if/elif/else ranges with no gaps or overlap. |
| Missing mark validation | Negative marks or marks >100 may be accepted and graded. | Validate type and range before assigning a grade. |
| Floating-point / rounding mismatch | A calculated percentage near a cutoff may round differently across layers. | Define whether grading uses raw or rounded values; apply one documented rule. |
| Null / non-numeric input | Conversion raises an exception or silently defaults to zero. | Reject invalid types and return a useful validation error. |
| Average versus total confusion | Grade assigned to total when policy expects percentage, or vice versa. | Separate total, maximum total, percentage, and grade variables. |

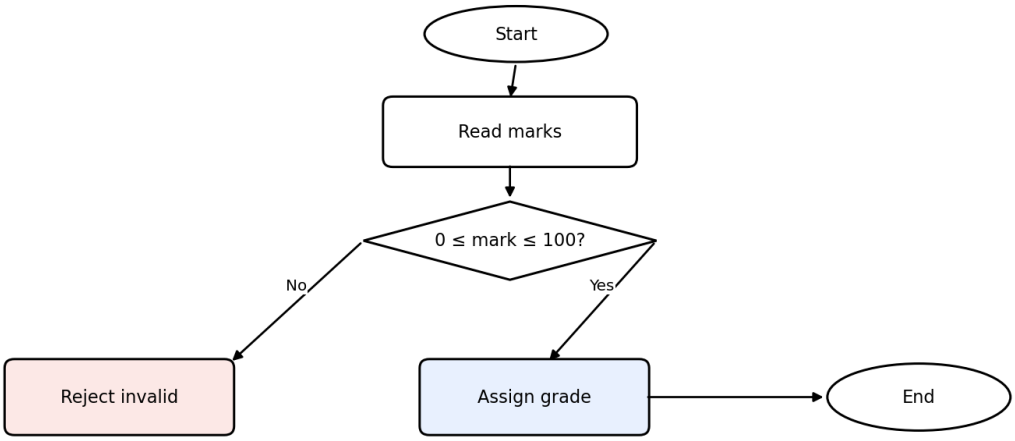
### b) ECP and BVA for marks

| Input class / boundary | Test values | Expected result |
|---|---|---|

| | | |
|---|---|---|
| Valid lowest / highest | 0, 100 | Accepted |
| Invalid lower / upper | -1, 101 | Rejected |
| Type errors | None, 'abc', 10.5 if integer-only policy | Rejected or handled according to documented policy |
| Grade A boundary | 89, 90, 91 | B, A, A |
| Grade B boundary | 79, 80, 81 | C, B, B |
| Grade C boundary | 69, 70, 71 | D, C, C |
| Grade D boundary | 59, 60, 61 | F, D, D |
| Empty input / multiple marks | [], [80, 90], [0,100] | Empty rejected unless supported; valid list calculates correctly. |

## c) CFG, independent paths, and cyclomatic complexity

**Q2 — Illustrative Control Flow Graph (CFG)**




For the simplified illustrative graph, the nodes are Start, Read marks, Validate range, Reject invalid, Assign grade, and End. With one binary decision, cyclomatic complexity is V(G)=D+1=2, so at least two independent paths are needed: valid input and invalid input. The actual complexity must be recalculated from the supplied source code because it was not included.

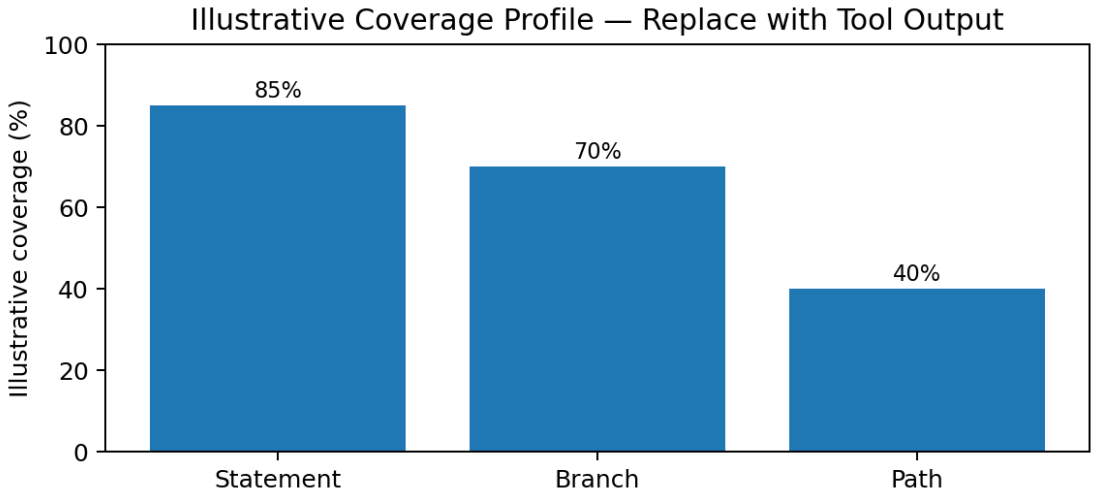
## d) JUnit tests and JaCoCo coverage

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class GradeCalculatorTest {
    @Test void zeroIsValidAndFails() {
        assertEquals("F", GradeCalculator.grade(0));
    }
    @Test void boundaryAtNinetyIsA() {
        assertEquals("A", GradeCalculator.grade(90));
    }
    @Test void boundaryBelowNinetyIsB() {
        assertEquals("B", GradeCalculator.grade(89));
    }
    @Test void oneHundredIsValid() {
        assertEquals("A", GradeCalculator.grade(100));
    }
    @Test void negativeMarkIsRejected() {
        assertThrows(IllegalArgumentException.class,
                     () -> GradeCalculator.grade(-1));
    }
    @Test void markAboveOneHundredIsRejected() {
        assertThrows(IllegalArgumentException.class,
                     () -> GradeCalculator.grade(101));
    }
}
```

Run the test suite through Maven, then generate JaCoCo reports. Example commands (assuming the project has the JaCoCo Maven plugin configured):

```
mvn clean test
mvn jacoco:report
# Open target/site/jacoco/index.html
```




Example values only; not measured from supplied source code.

The chart uses synthetic values solely to show how coverage should be presented. It is not a JaCoCo report. Statement coverage alone is insufficient: a test can execute both sides of a decision incorrectly or fail to exercise threshold boundaries. Review missed lines and branches, then add targeted tests.

### e) Mutation testing and adequacy

| Mutation | Test expected to kill it |
|---|---|
| Change mark >= 90 to mark > 90 | Input 90 must still receive A. |
| Change lower bound 0 to 1 | Input 0 must be accepted. |
| Remove mark > 100 validation | Input 101 must be rejected. |
| Change grade F upper bound 59 to 58 | Input 59 must receive F. |
| Replace computed percentage with raw total | Multi-subject case with known percentage detects mismatch. |

Use a mutation tool supported by the project (for example, PIT for Java). A surviving mutant means either a missing test or an equivalent mutation; investigate rather than treating the mutation score as a standalone quality guarantee. Release only when all specified boundaries, invalid inputs, and critical branches pass.

## Q3. Error Analysis of an Online Banking Application

### a) Request/response analysis

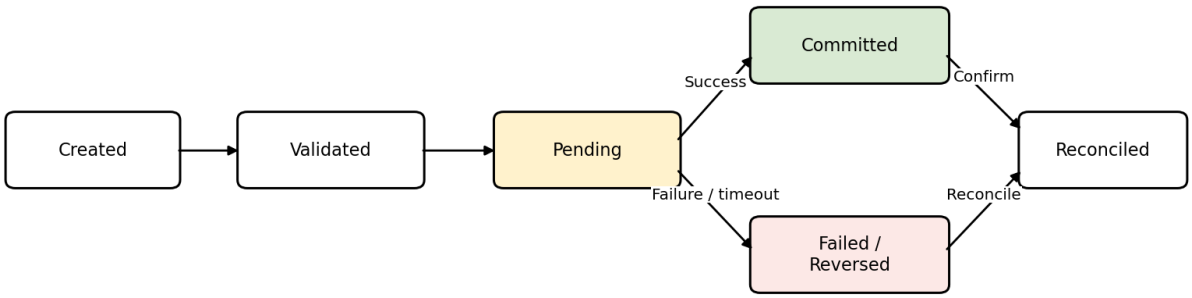
| Defect ID | Expected API behavior | Possible actual behavior | Likely cause / fix |
|---|---|---|---|
| BANK-01 invalid amount | Reject zero, negative, malformed, or over-limit amounts with documented 4xx response. | Transfer accepted or generic 500 returned. | Server-side schema/business validation; consistent error mapping. |
| BANK-02 insufficient funds | Reject atomically; balance and ledger remain consistent. | Debit without credit or inconsistent balance. | Non-atomic updates; use database transaction and ledger invariants. |
| BANK-03 timeout after commit | Return/recover one definitive transaction outcome; retries must not duplicate transfer. | Client retries and creates duplicate transfer. | Idempotency key, durable transaction ID, status lookup and reconciliation. |
| BANK-04 inconsistent status | Statuses follow a defined lifecycle and are monotonic where appropriate. | UI says failed while ledger shows committed. | Single source of truth, event ordering, reconciliation. |
| BANK-05 unauthorized request | Reject invalid/expired token and prevent cross-account access. | Request accepted or leaks account details. | Authentication and object-level authorization checks. |

## b) Black-box techniques

| Technique | Test data / combinations | Expected |
|---|---|---|
| ECP: amount | Invalid (≤0, malformed); valid positive; over daily limit | Reject invalid/over-limit; accept valid within policy. |
| BVA: amount | 0.00, minimum allowed, just below minimum, maximum allowed, just above maximum | Enforce exact documented min/max. |
| Random testing | Random amounts, missing fields, long strings, Unicode, malformed JSON, duplicate idempotency keys | No crash, bypass, double debit, or inconsistent ledger. |
| Decision table | Authenticated × beneficiary valid × sufficient funds × amount valid × limit not exceeded | Commit only when all required conditions pass. |
| Repeated requests | Same idempotency key and payload; same key with changed payload | Same result for exact retry; reject conflicting payload reuse. |

## c) Use-case, state-transition, and pairwise testing

**Q3 — Fund Transfer Transaction State Model**




| Use case | Steps | Expected result |
|---|---|---|
| Successful transfer | Authenticate → choose verified beneficiary → valid amount → confirm | One debit and one credit; status committed. |
| Invalid transfer | Authenticate → amount = 0 / insufficient balance | 4xx validation/business error; no ledger change. |
| Network loss | Submit transfer, drop response, query status, retry with same key | One transfer only; status query resolves outcome. |
| Token expiry | Submit with expired token | 401/403 according to API contract; no transfer. |
| Duplicate click | Submit same request twice rapidly | Idempotent result; one transaction. |

Pairwise dimensions should include authentication state (valid/expired/invalid), transaction amount (below/minimum/normal/maximum/above), network (stable/timeout/disconnect), browser/client (supported variants), and retry pattern (none/same key/different key). Use a pairwise generator to cover every pair of values, then add risk-based tests for three-way combinations such as timeout + retry + server commit.

## d) Postman execution and evidence template

```
POST /api/v1/transfers
Authorization: Bearer <TEST_TOKEN>
Idempotency-Key: <UNIQUE_TEST_KEY>
Content-Type: application/json

{
  "sourceAccountId": "TEST-ACCOUNT-A",
  "beneficiaryId": "TEST-BENEFICIARY-B",
  "amount": 125.00,
  "currency": "INR"
}
```

| Test ID | Request variation | Expected response | Evidence to record |
|---|---|---|---|
| API-01 | Valid request | 201/202 per contract; transaction ID returned | Status, body, request ID, ledger check |
| API-02 | amount = 0 | 400/422; no transaction created | Error code/message and unchanged balances |

| | | | |
|---|---|---|---|
| API-03 | amount above limit | 400/422; limit error | Response body and limit configuration |
| API-04 | Expired token | 401/403; no state change | Status, response headers/body |
| API-05 | Timeout then retry same key | One transaction, stable transaction ID/status | Both responses plus transaction lookup |
| API-06 | Same key but different payload | 409 or documented idempotency conflict | Response and absence of second transfer |

The table is an execution template, not captured Postman output. Export the Postman collection/run report and redact tokens, account identifiers, and personal data.

### e) Adequacy and unavailable white-box measures

- Maintain requirement-to-test traceability for authentication, validation, funds, limits, idempotency, status lifecycle, audit trail, and reconciliation.
- Risk coverage should prioritize duplicate transfers, unauthorized access, inconsistent ledger state, and ambiguous outcomes after network failure.
- Without source code or instrumentation, internal statement, branch, condition, path, and data-flow coverage cannot be calculated reliably. API response coverage and requirement coverage are not substitutes for code coverage.
- Use contract tests, schema validation, negative tests, security tests, resilience tests, and independent ledger reconciliation.

## Q4. Error Analysis of a COTS Learning Management System

### a) Observable behavior / reverse engineering

With no source code, reverse engineering means documenting externally observable inputs, outputs, states, browser/network behavior, and vendor-documented rules—not reconstructing internal implementation. Build a black-box model for login, course registration, assignment upload, and result viewing.

| Area | Expected behavior | Likely error / diagnostic |
|---|---|---|
| Login | Valid user logs in; invalid credentials rejected; session expires safely. | Session cookie issues, redirect loop, insecure caching, inconsistent error. |
| Course registration | Eligible user registers once; capacity and prerequisites enforced. | Duplicate enrollment, stale capacity, race conditions. |
| Assignment upload | Supported file/size accepted; invalid file rejected; submission receipt displayed. | Upload timeout, MIME/extension mismatch, lost confirmation, duplicate submission. |
| Result viewing | Only authorized student sees correct published result. | Cross-user data exposure, stale result, incorrect cache. |

### b) ECP, BVA, exploratory testing, error guessing

| Target | Tests | Expected |
|---|---|---|
| Upload file size | 0 bytes, minimum, max−1, max, max+1 | Apply documented size limit consistently. |
| File type | Allowed extension/MIME; disallowed executable; mismatched extension and MIME | Accept only approved types; reject unsafe/mismatched types. |
| Network | Stable, slow, disconnect mid-upload, reconnect | Clear error/retry behavior; no corrupt or duplicate final submission. |
| Course capacity | One seat left; simultaneous two registrations; full course | Never exceed capacity; one confirmed enrollment per student/course. |
| Browser compatibility | Each vendor-supported browser/version/OS combination | Same core functionality and understandable errors. |
| Exploratory/error guessing | Double click, back/refresh, expired session, long filename, special characters, interrupted upload | No silent data loss, unauthorized access, or duplicate records. |

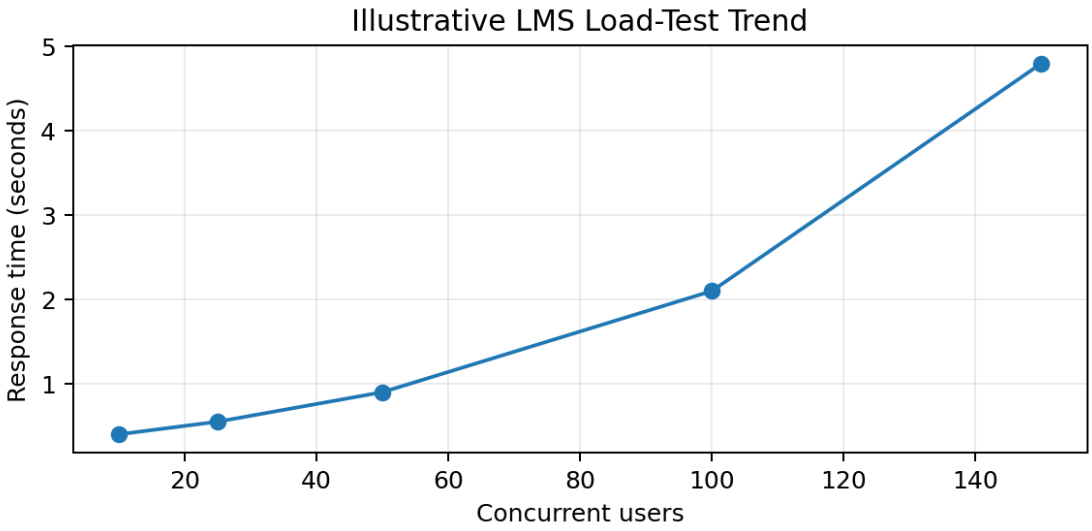
### c) Selenium functional checks and JMeter performance checks

Selenium should verify observable workflows across the supported browser matrix. JMeter should model realistic login/course browsing/upload/result-viewing traffic, ramp users gradually, and capture latency percentiles, throughput, error rate, and server resource indicators where available.

```
# Selenium sketch: assert upload confirmation (adapt to real LMS)
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
try:
    driver.get("https://TEST-LMS.example.edu")
    # Perform test-account login using environment-provided credentials.
    # Navigate to the course and assignment upload form.
    driver.find_element(By.ID, "assignment-file").send_keys(
        "/path/to/controlled-test-file.pdf"
    )
    driver.find_element(By.ID, "submit-assignment").click()
    confirmation = WebDriverWait(driver, 15).until(
        EC.visibility_of_element_located((By.ID, "submission-receipt"))
    )
    assert confirmation.is_displayed()
    driver.save_screenshot("Q4_upload_confirmation.png")
finally:
    driver.quit()
```




Synthetic example only; acceptance threshold must be agreed in advance.

Illustrative performance acceptance criteria (confirm with stakeholders before execution): at 100 concurrent users, p95 page response ≤2.0 s, error rate <1%, and assignment uploads up to the documented size complete within 10 s on the agreed test network. These values and the plotted data are synthetic examples, not measured results.

## d) COTS testing limitations

- Without source code, internal CFGs, independent paths, statement/branch coverage, and internal data-flow coverage normally cannot be measured.
- Vendor instrumentation or a source-level coverage build could make some measures available, but access must be explicitly provided and supported.
- Black-box tests can still provide strong evidence through requirements coverage, use-case coverage, browser compatibility, API/network behavior, performance metrics, security tests, and reproducible defect reports.
- Vendor-controlled updates can change behavior; record build/version, configuration, browser version, test date, and release notes for every run.

## e) Acceptance evaluation

| Quality dimension | Measure | Acceptance decision |
|---|---|---|
| Functional | All critical login, registration, upload, and results tests pass. | Reject release for any critical functional/security failure. |
| Compatibility | Core workflow passes on every supported browser/OS combination. | Block release for unsupported behavior within declared support matrix. |
| Performance | p95 latency, throughput, error rate, upload completion under agreed load. | Accept only if predefined thresholds are met. |
| Reliability | Interrupted uploads, retries, duplicate submissions, recovery. | No silent loss, corruption, or unintended duplicate submission. |

| | | |
|---|---|---|
| Risk | Severity × likelihood × exposure; documented vendor limitations. | Obtain vendor remediation or formally accept only low residual risks. |

# Q5. Error Analysis of a Student Result-Processing Program

## a) Logic errors: conditions, loops, definitions, and uses

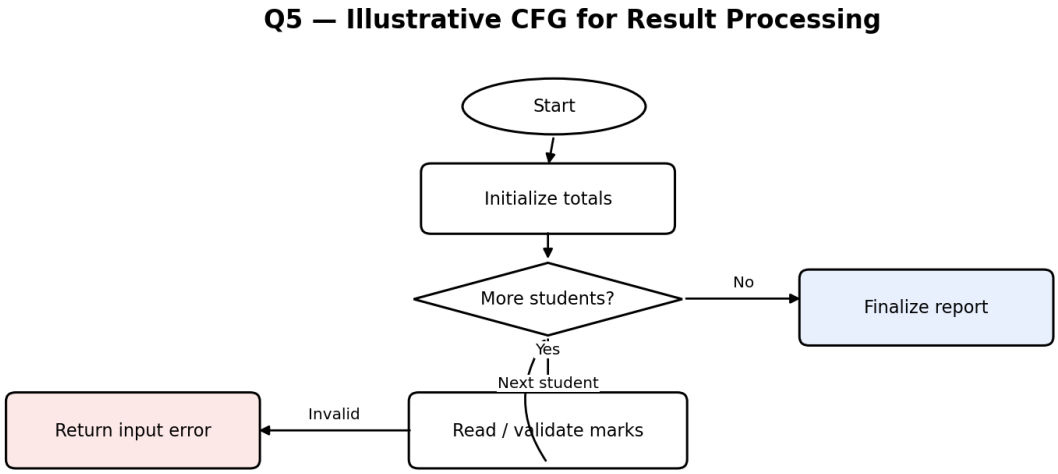
The source program is not attached, so these are defects to check rather than confirmed defects. Trace every variable from initialization through update and output.

| Potential error | Symptom / root cause | Correction |
|---|---|---|
| Accumulator not initialized per student | One student's marks contaminate the next student's total. | Set total = 0 at the start of each student iteration. |
| Loop off-by-one | First/last student or subject skipped, or index out of range. | Use explicit ranges and test 0, 1, and maximum counts. |
| Wrong loop nesting | Subjects are counted as students or totals are mixed across students. | Separate outer student loop and inner subject loop. |
| Invalid mark accepted | Negative or >100 mark changes average/grade. | Validate every mark before adding it to the total. |
| Incorrect pass rule | Overall average used when every subject must pass, or vice versa. | Encode the institution's exact pass rule and test both interpretations. |
| Uninitialized / stale variable | Grade or pass flag carries over from prior student. | Initialize per record and assign on every path. |
| Integer division / rounding | Average truncates near threshold. | Use correct numeric type and documented rounding policy. |

## b) Boundary-value, equivalence-class, and error-guessing tests

| Input | Representative tests | Expected behavior |
|---|---|---|
| Marks | -1, 0, 1, pass mark−1, pass mark, pass mark+1, 100, 101 | Reject out-of-range; exact pass-boundary behavior follows policy. |
| Student count | -1, 0, 1, maximum supported, maximum+1 | Reject negative/over-limit; handle zero according to specification. |
| Subject count | 0, 1, configured maximum, maximum+1 | Reject unsupported counts; avoid division by zero. |
| Dataset | One student, multiple students with different totals, empty input | Totals remain independent and output order is correct. |
| Error guessing | Missing mark, duplicate student ID, malformed row, extra column, all-zero marks | Clear validation errors; no silent defaulting or cross-record contamination. |

## c) CFG, cyclomatic complexity, independent paths



For the simplified graph shown, decisions include the student-loop condition and input-validity decision. The exact cyclomatic complexity is V(G)=E−N+2P (for one connected component, P=1), or decisions + 1 for a simple structured graph. Count decisions in the actual implementation—including nested conditions and loop exits—before stating the final number. Test at least: zero students, one valid student, multiple valid students, invalid mark, and loop termination at the final student.

## d) pytest, Coverage.py, data-flow, mutation testing

```
# Example contract for a result-processing function
def test_single_student_total_and_grade():
    result = process_results([{"id": "S1", "marks": [80, 90]}])
    assert result[0]["total"] == 170
    assert result[0]["average"] == 85
    assert result[0]["grade"] == "B"

def test_rejects_mark_above_100():
    import pytest
    with pytest.raises(ValueError):
        process_results([{"id": "S1", "marks": [80, 101]}])

def test_student_totals_do_not_leak():
    result = process_results([
        {"id": "S1", "marks": [100, 100]},
        {"id": "S2", "marks": [0, 0]},
    ])
    assert result[0]["total"] == 200
    assert result[1]["total"] == 0

def test_empty_subject_list_is_handled():
    import pytest
    with pytest.raises(ValueError):
        process_results([{"id": "S1", "marks": []}])
```

```
python -m pytest -q
coverage run --branch -m pytest
coverage report -m
coverage html
# Open htmlcov/index.html and inspect uncovered lines/branches.
ruff check .
# Or use: pylint your_module.py
```

Data-flow testing should inspect definition-use pairs: total initialized → incremented → read for average; average assigned → read for grade; pass flag initialized/updated → read for final decision. Look for use-before-definition, stale values, and definitions that never reach a use. Mutation testing should alter comparison operators, remove validation, change loop limits, or omit accumulator resets; the tests should fail for each meaningful mutation.

## e) Adequacy and release recommendation

| Measure | What to evaluate | Release expectation |
|---|---|---|
| Statement coverage | Executed statements / executable statements. | High coverage is useful but not sufficient; inspect all uncovered code. |
| Branch coverage | True/false outcomes of each decision. | Cover both outcomes for validation, pass/fail, and loop conditions. |
| Path considerations | Independent paths and loop boundaries. | Cover feasible independent paths; exhaustive path testing may be impractical with loops. |
| Data-flow coverage | Critical definition-use pairs. | Every important total/average/grade/pass variable has tested definition-use paths. |
| Mutation results | Mutants killed / non-equivalent mutants. | Investigate surviving mutants, especially at boundaries and validation. |
| Static analysis | Ruff/Pylint findings, type/undefined-variable issues. | Resolve correctness and high-severity findings before release. |

Recommendation: release only after all high-risk defects are fixed, boundary and invalid-input tests pass, results are independently recalculated for representative datasets, and coverage/mutation reports are reviewed. Do not use a single coverage percentage as proof of correctness.

## Overall conclusion

Across all five scenarios, the most reliable error-analysis process is to define expected behavior first, partition the input domain, test boundaries, model decisions and states, execute repeatable cases, preserve evidence, trace tests to requirements, and prioritize defects by impact and likelihood. White-box measures (CFG, independent paths, statement/branch and data-flow coverage) require source code or suitable instrumentation. For systems without source access, use observable functional, API, compatibility, performance, security, and requirement coverage while clearly documenting the limits.

## Submission checklist

- Fill in name, register number, date, and student declaration/signature on the institution's original cover sheet if required.
- Replace illustrative/synthetic test results and charts with actual output from the specified tools before claiming execution.
- Confirm the real attendance policy, grade boundaries, bank API contract, LMS acceptance thresholds, and result-processing rules with the appropriate specification.
- Attach sanitized real screenshots, logs, JUnit/JaCoCo reports, Postman collection-run output, JMeter report, and pytest/Coverage.py report where available.